

Software Requirements Specification

A Mobile Application for Facility Management

Team

The smart cloud

Members

Nathalie Romani I 13003489

Mariam Ali | 13003733

Maria Ramy | 13002733

Kenzy Sameh | 13001092

Malak Kotb | 13001812

Zeinab ahmed | 1300

Tutorial Details

T3 | Moamen Atia

1. Introduction	3
1.1 Product Vision & Scope	3
1.2 Definitions and Acronyms	3
2. Functional Requirements	3
2.1 User Stories	7
2.2 UML Use-case Diagram	11
3. Non-functional Requirements:	11
4. Constraints	12
5. ERD: Entity Relationship Diagram	12
6. Technology Stack	12
7. Future Scope of the Project	12

1. Introduction

This Software Requirements Specification (SRS) describes the requirements for a Facility Management mobile application for the German International University (GIU). The app enables GIU community members to report infrastructure, maintenance, sustainability, and cleanliness issues, and enables the Facility Management team to track and resolve them through status updates. This document focuses on a minimum viable product (MVP) that delivers the essential closed-loop workflow: submit an issue → update status → view resolution.

1.1 Product Vision & Scope

Product Vision: Provide a fast, simple, and formal channel for reporting campus facility issues and ensuring they are handled transparently. Scope (MVP): authenticated users can submit tickets with a photo, short description, and location; all users can view tickets and their statuses; the Facility Management team can update statuses (Submitted → In Progress → Resolved). Out of Scope for MVP (Future): assigning individual workers, worker notifications, advanced administration, analytics, and SLA tracking.

1.2 Definitions and Acronyms

Definitions & Acronyms used in this document:

SRS: Software Requirements Specification

MVP: Minimum Viable Product

Ticket: A digital record representing a reported facility issue

UML: Unified Modeling Language

2. Functional Requirements

1. Community Member Functional Requirements

FR-CM-01: Account Creation

The system shall allow a Community Member to create an account by providing required information (e.g., name, email, and password).

FR-CM-02: User Authentication

The system shall allow a Community Member to log in using a valid email and password.

FR-CM-03: Issue Submission

The system shall allow a Community Member to submit an issue including the following details:

- Location
- Photo
- Category
- Description

FR-CM-04: Issue Description

The system shall allow a Community Member to add a textual description when submitting an issue.

FR-CM-05: View Submitted Issues

The system shall allow a Community Member to view a list of issues they have submitted.

FR-CM-06: Logout

The system shall allow a Community Member to log out of the application.

FR-CM-07: View Issue Status

The system shall display the current status of an issue to the Community Member.
Possible statuses include:

- Pending
- In Progress
- Resolved

2. Facility Manager Functional Requirements

FR-FM-01: Account Creation

The system shall allow a Facility Manager to create an account.

FR-FM-02: User Authentication

The system shall allow a Facility Manager to log in using a valid email and password.

FR-FM-03: Logout

The system shall allow a Facility Manager to log out of the application.

FR-FM-04: View All Issues

The system shall allow a Facility Manager to view all submitted issues in the system.

FR-FM-05: Update Issue Status

The system shall allow a Facility Manager to update the status of an issue.

FR-FM-06: Assign Issue to Worker

The system shall allow a Facility Manager to assign an issue to a specific worker.

FR-FM-07: Close Issue

The system shall allow a Facility Manager to close an issue once it has been resolved.

3. Worker Functional Requirements

FR-W-01: Account Creation

The system shall allow a Worker to create an account.

FR-W-02: User Authentication

The system shall allow a Worker to log in using a valid email and password.

FR-W-03: Logout

The system shall allow a Worker to log out of the application.

FR-W-04: View Assigned Issues

The system shall allow a Worker to view issues assigned to them.

FR-W-05: Comment on Issue

The system shall allow a Worker to add comments to an issue.

FR-W-06: Update Issue Status to In Progress

The system shall allow a Worker to mark an assigned issue as **In Progress**.

FR-W-07: Upload Completion Photo

The system shall allow a Worker to upload a photo after completing the work on an issue.

4. System Admin Functional Requirements (Optional)

FR-SA-01: Admin Authentication

The system shall allow a System Admin to log in.

FR-SA-02: Logout

The system shall allow a System Admin to log out.

FR-SA-03: Account Management

The system shall allow a System Admin to activate or deactivate user accounts.

FR-SA-04: View Registered Users

The system shall allow a System Admin to view all registered users in the system.

Product BackLog

Backlog ID	User Story ID	Priority
Core System Features		
PB-01	US-CM-01	High
PB-02	US-CM-02	High
PB-03	US-CM-03	High
PB-04	US-CM-04	High
PB-05	US-CM-05	High
PB-06	US-CM-07	High
PB-07	US-FM-01	High
PB-08	US-FM-02	High
PB-09	US-W-01	High
PB-10	US-W-02	High
Issue Management Features		
PB-11	US-FM-04	High

PB-12	US-FM-06	High
PB-13	US-W-04	High
PB-14	US-W-06	High
PB-15	US-W-05	Medium
PB-16	US-W-07	Medium
PB-17	US-FM-05	High
PB-18	US-FM-07	High
Authentication & Security		
PB-19	US-CM-06	Medium
PB-20	US-FM-03	Medium
PB-21	US-W-03	Medium
System Administration (Optional)		
PB-22	US-SA-01	Low
PB-23	US-SA-02	Low
PB-24	US-SA-03	Low
PB-25	US-SA-04	Low

2.1 User Stories

1. Community Member

US-CM-01:

As a Community Member, I should be able to create an account so that I can log in to the application.

US-CM-02:

As a Community Member, I should be able to log in using my email and password so that I can access the application

US-CM-03:

As a Community Member, I should be able to submit an issue with its details (location, photo, and category) so that the problem can be reported accurately.

US-CM-04:

As a Community Member, I should be able to add a description to the issue so that the worker understands the problem clearly.

US-CM-05:

As a Community Member, I should be able to view my submitted issues so that I can track their status.

US-CM-06:

As a Community Member, I should be able to log out so that my data remains secure on shared devices.

US-CM-07

As a Community Member, I should be able to view the status of my issue (Pending, In Progress, Resolved) so that I know the progress of my request.

2. Facility Manager**US-FM-01:**

As a Facility Manager, I should be able to create an account so that I can log in to the application.

US-FM-02:

As a Facility Manager, I should be able to log in using my email and password so that I can access the application.

US-FM-03:

As a Facility Manager, I should be able to log out so that my data remains secure on shared devices.

US-FM-04:

As a Facility Manager, I should be able to view all submitted issues so that I can assign them to workers.

US-FM-05:

As a Facility Manager, I should be able to update an issue's status so that the system reflects the current state of the issue.

US-FM-06

As a Facility Manager, I should be able to assign an issue to a worker so that the worker can start fixing it.

US-FM-07

As a Facility Manager, I should be able to close an issue after it is resolved so that the system reflects that the issue is completed.

3. Worker

US-W-01:

As a Worker, I should be able to create an account so that I can log in to the application.

US-W-02:

As a Worker, I should be able to log in using my email and password so that I can access the application.

US-W-03:

As a Worker, I should be able to log out so that my data remains secure on shared devices.

US-W-04:

As a Worker, I should be able to view my assigned issues so that I can start working on them.

US-W-05:

As a Worker, I should be able to comment on an issue to indicate that the work is completed so that the facility manager can update its status.

US-W-06

As a Worker, I should be able to mark an issue as "In Progress" so that the facility manager and community member know that it is being worked on.

US-W-07

As a Worker, I should be able to upload a photo after fixing the issue so that the facility manager can verify the work.

4. System Admin (optional for Now):

US-SA-01:

As a System Admin, I should be able to log in so that I can access the application.

US-SA-02:

As a System Admin, I should be able to log out so that my data remains secure on shared devices.

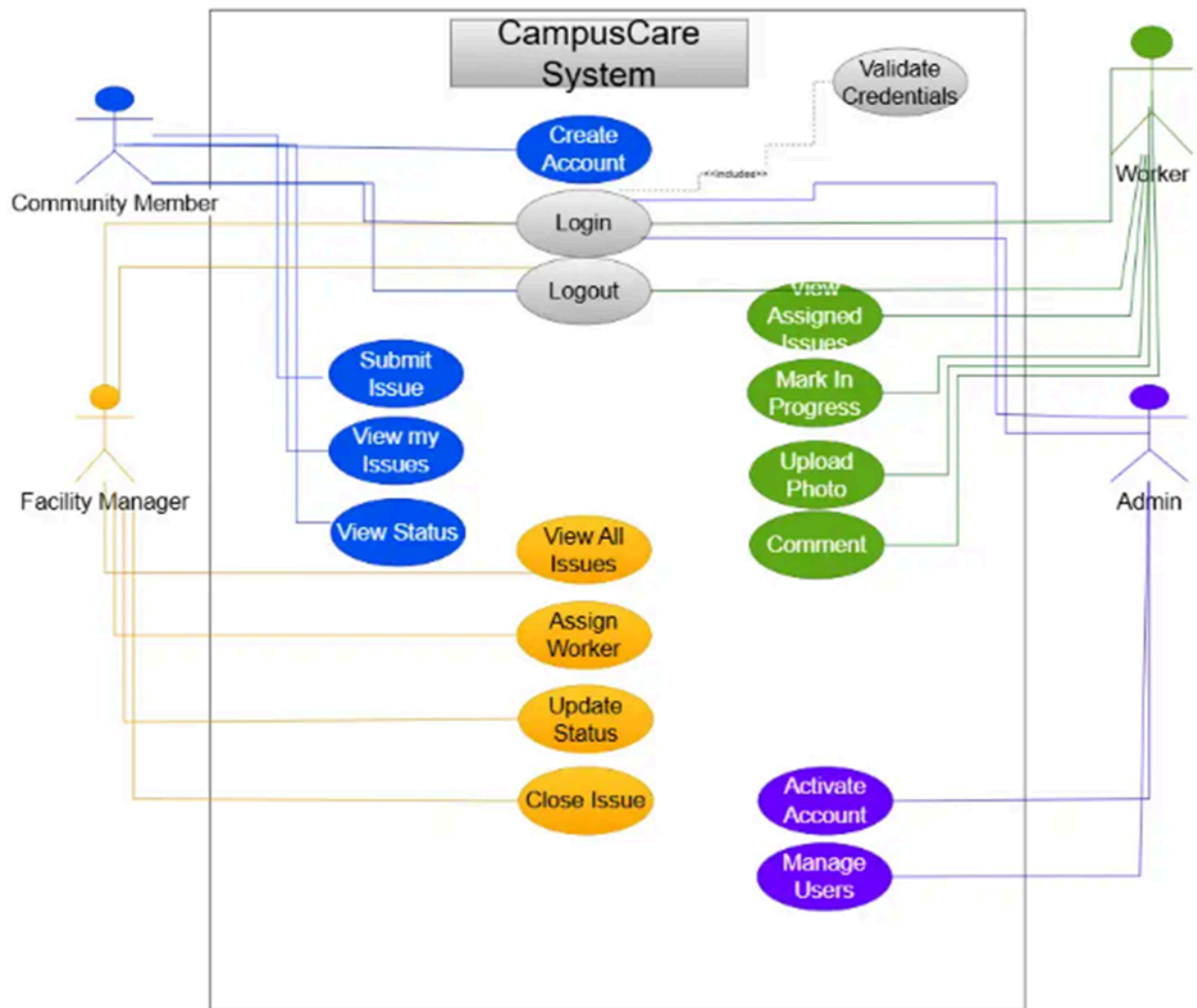
US-SA-03:

As a System Admin, I should be able to activate or deactivate any account so that I can ensure all users follow the system rules.

US-SA-04

As a System Admin, I should be able to view all registered users so that I can manage the system users.

2.2 UML Use-case Diagram



3. Non-functional Requirements:

Non-functional requirements define quality attributes for performance, usability, security, and reliability.

NFR-01: Performance: Core screens (login, tickets list, ticket details) should load within 3 seconds on a typical campus network.

- NFR-02: Scalability: The backend should support at least 500 concurrent users during peak campus hours.
- NFR-03: Security: Passwords must be stored using secure hashing; APIs must validate inputs and implement basic rate limiting.

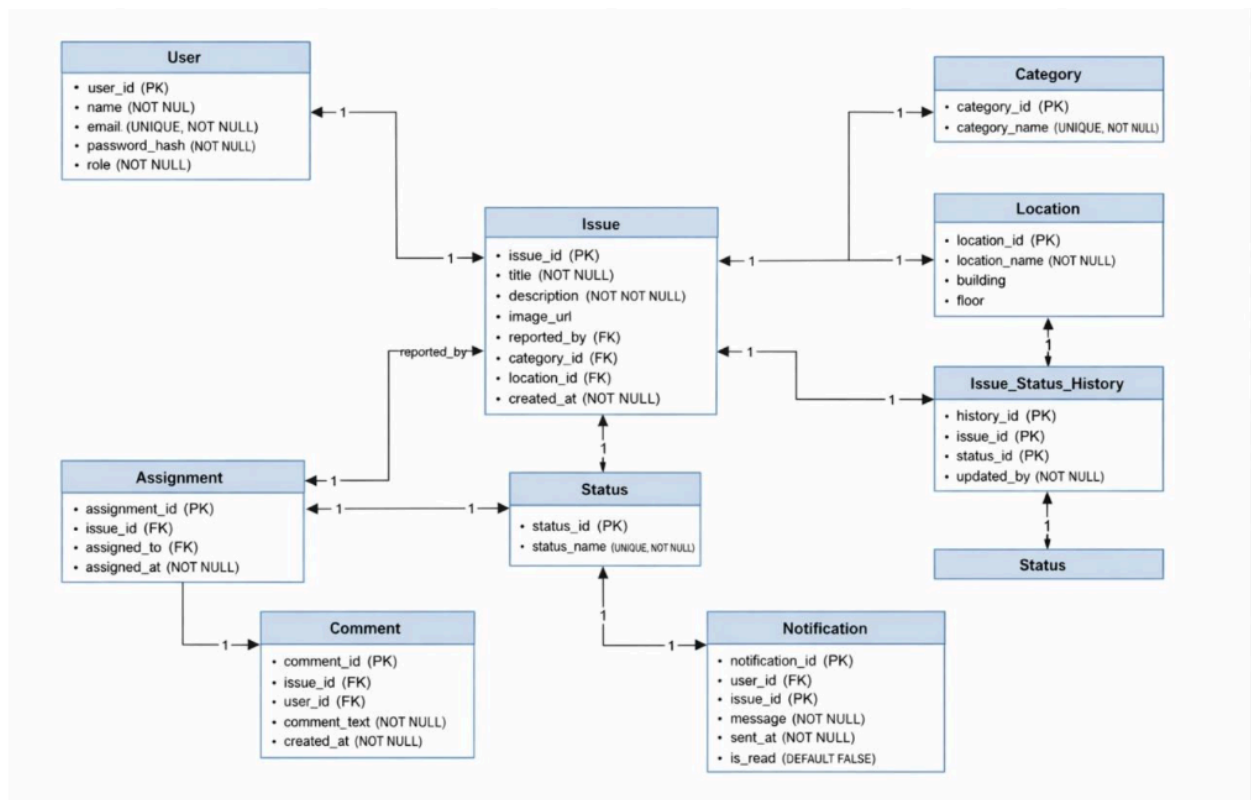
- NFR-04: Reliability: Image uploads must be resilient to unstable mobile connections (retry and clear error messages).
- NFR-05: Usability: A ticket should be creatable in under 1 minute (photo → location → description → submit).
- NFR-06: Availability & Recovery: Daily automated database backups and the ability to restore within 24 hours.

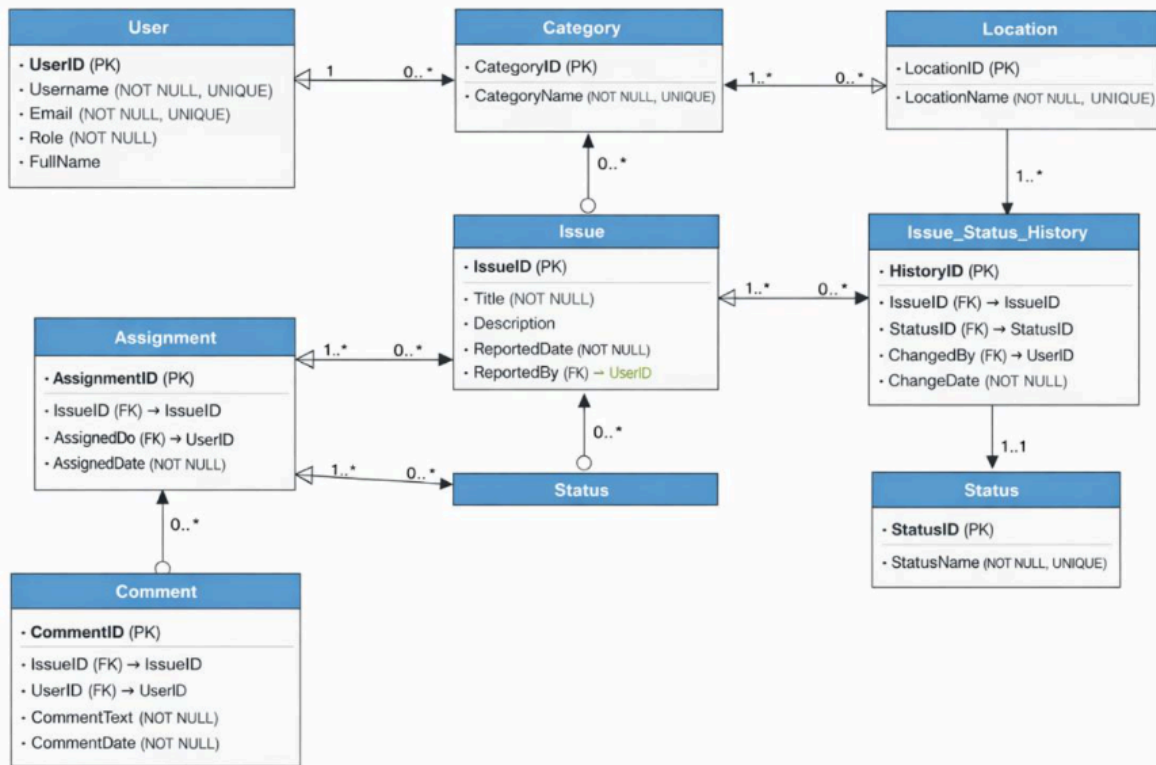
4. Constraints

Constraints:

- Mobile app must be developed using React Native with Expo.
- Backend must be implemented using Node.js with RESTful APIs.
- Database must be PostgreSQL.
- Ticket images must be stored using a cloud-based object storage service.
- Project schedule and team resources may limit the MVP to the essential workflow only (no worker assignment in MVP).

5. ERD: Entity Relationship Diagram





6. Technology Stack

- Mobile: React Native (Expo) — UI, navigation, camera/gallery access for ticket photos.
- Backend: Node.js (Express) — RESTful APIs for authentication, tickets CRUD, and status updates.
- Database: PostgreSQL — stores users, tickets, and statuses.
- Storage: Cloud object storage (e.g., S3-compatible) — stores ticket images and returns secure URLs.

7. Future Scope of the Project

- Add Worker role: facility team assigns tickets to workers; workers view assigned tasks.

- Ticket priority, categories (maintenance/cleanliness/sustainability)
- Search, filters, and basic analytics dashboard (open vs resolved, average resolution time).